3. Tabla CLR(1) ACTION/GOTO

Construcción

- Si GOTO$(I_i, t) = I_j$ y t es terminal, colocar `sj` en ACTION[i,t].
- Si $[A \to \alpha\cdot, a] \in I_i$ y la producción es pk ≠p0, colocar `rk` solo en ACTION[i,a].
- Si [`Start` → `Program`·, \$] $\in I_i$, colocar ACCEPT en ACTION[i,\$]. No generar r0.
- Para cada transición con no terminal A, colocar su destino en GOTO[i,A]. Reunir acciones candidatas antes de comprobar conflictos; no sobrescribirlas.

Resultado: 509 SHIFT, 795 REDUCE, una ACCEPT y 221 entradas GOTO. Cero conflictos shift/reduce y reduce/reduce. ACTION tiene 32 columnas; GOTO usa los 18 no terminales originales, sin Start.

Significado de las acciones y del árbol

- **SHIFT sj:** reconoce el token actual y pasa al estado j. Ese token corresponde a una hoja del árbol, con su lexema, línea y columna.
- **REDUCE rk:** reconoce la parte derecha de la producción pk como una instancia de su no terminal izquierdo. El siguiente token no se consume. La producción determina el padre y el orden de sus hijos en el árbol.
- **GOTO:** determina la transición con el no terminal reconocido mediante una reducción.
- **Producción vacía:** reconoce una parte derecha sin símbolos. Su nodo no tiene hijos y no consume tokens.
- **ACCEPT:** confirma que Program se reconoció completo y se alcanzó el fin de entrada. Program es la raíz del árbol sintáctico.
- **Celda ACTION ausente:** no existe una acción válida para ese estado y token; corresponde a un error sintáctico.

Las producciones intermedias también forman parte del árbol. Así se conserva la relación entre cada estructura reconocida y sus reglas semánticas. Para una producción no vacía, la posición del nodo corresponde al primer hijo; para una vacía, a la posición del siguiente token.

Ejemplos: `ACTION[0,int]=s2`; `ACTION[28,;]=r12`; `GOTO[28,Init]=50`; `ACTION[1,$]=acc`. En r12 se reconoce Init vacío y se conserva `;` como siguiente token.

Ejemplo de ejecución: declaración sin inicializador

La siguiente traza corresponde a `int main(){int x;}`. Inicia cuando ya se reconocieron la cabecera, la llave y la lista Items vacía. En cada fila se muestra la situación *antes* de ejecutar la acción.

Pila de estados	Siguiente	Acción y efecto
`[0,2,3,4,5,7,8]`	`int`	`s22`: Consume int.
`[0,2,3,4,5,7,8,22]`	`ID`	`r8`: Type -> int; GOTO(8,Type)=17.
`[0,2,3,4,5,7,8,17]`	`ID`	`s28`: Consume ID, cuyo lexema es x.
`[0,2,3,4,5,7,8,17,28]`	`;`	`r12`: Init -> epsilon; GOTO(28,Init)=50.
`[0,2,3,4,5,7,8,17,28,50]`	`;`	`s87`: Consume el punto y coma.
`[0,2,3,4,5,7,8,17,28,50,87]`	`}`	`r7`: Decl -> Type ID Init ;. Retira cuatro estados y consulta GOTO(8,Decl)=11.

En r12 no se retira nada de las pilas y tampoco se consume `;`. Por eso la fila siguiente vuelve a consultar ese mismo token. En r7 se retiran Type, ID, Init y `;`; se crea un nodo Decl con esos cuatro hijos y se conserva el lexema de x en su hoja.

El final del análisis reduce Block con p2 y Program con p1. Se llega a la pila `[0,1]` con el token `$`; ACTION[1,$] permite ACCEPT. El árbol resultante tiene Program como raíz.

Interpretar una fila de la tabla

La entrada `r4 : ;, ID, char, ...` en I7 significa que cada uno de esos terminales tiene la misma reducción p4. No significa que se consuman todos. La entrada `Items : 8` está en GOTO, porque Items es un no terminal obtenido al reducir.

Un conflicto shift/reduce existiría si una misma celda pidiera consumir el token y reducir. Un conflicto reduce/reduce existiría si dos producciones distintas pidieran reducir en esa celda. Registrar dos veces la misma acción no crea un conflicto. La tabla entregada no tiene celdas con acciones distintas en competencia.

Tabla completa

Cada par `acción : {terminales}` define esa acción en todas las columnas enumeradas de la fila. No agrega acciones a otras columnas.

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
0	s2 : int	Program: 1
1	acc : $	–
2	s3 : main	–
3	s4 : (	–
4	s5 :)	–
5	s7 : {	Block: 6
6	r1 : $	–
7	r4 : ;, ID, char, float, if, int, return, while, {, }	Items: 8
8	s12 : ID; s19 : char; s20 : float; s21 : if; s22 : int; s23 : return; s24 : while; s25 : {; s26 : }; s9 : ;	Block: 10; Decl: 11; Item: 13; Matched: 14; Simple: 15; Stmt: 16; Type: 17; Unmatched: 18
9	r24 : ;, ID, char, float, if, int, return, while, {, }	–
10	r16 : ;, ID, char, float, if, int, return, while, {, }	–
11	r5 : ;, ID, char, float, if, int, return, while, {, }	–
12	s27 : =	–
13	r3 : ;, ID, char, float, if, int, return, while, {, }	–
14	r13 : ;, ID, char, float, if, int, return, while, {, }	–
15	r15 : ;, ID, char, float, if, int, return, while, {, }	–
16	r6 : ;, ID, char, float, if, int, return, while, {, }	–
17	s28 : ID	–
18	r14 : ;, ID, char, float, if, int, return, while, {, }	–
19	r10 : ID	–
20	r9 : ID	–
21	s29 : (	–
22	r8 : ID	–
23	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Equality: 36; Expr: 37; Mul: 41; Primary: 42; Relation: 43; Unary: 45
24	s46 : (	–
25	r4 : ;, ID, char, float, if, int, return, while, {, }	Items: 47
26	r2 : $	–
27	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Equality: 36; Expr: 48; Mul: 41; Primary: 42; Relation: 43; Unary: 45
28	r12 : ;; s49 : =	Init: 50
29	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 58; Mul: 62; Primary: 63; Relation: 64; Unary: 66

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
30	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 67
31	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 68; Mul: 62; Primary: 63; Relation: 64; Unary: 66
32	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 69
33	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 70
34	r33 : !=, ;, <, <=, ==, >, >=; s71 : +; s72 : -	–
35	r48 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
36	r25 : ;; s73 : !=; s74 : ==	–
37	s75 : ;	–
38	r47 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
39	r45 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
40	r46 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
41	r36 : !=, +, -, ;, <, <=, ==, >, >=; s76 : %; s77 : *; s78 : /	–
42	r44 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
43	r28 : !=, ;, ==; s79 : <; s80 : <=; s81 : >; s82 : >=	–
44	r49 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
45	r40 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
46	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 83; Mul: 62; Primary: 63; Relation: 64; Unary: 66
47	s12 : ID; s19 : char; s20 : float; s21 : if; s22 : int; s23 : return; s24 : while; s25 : {; s84 : }; s9 : ;	Block: 10; Decl: 11; Item: 13; Matched: 14; Simple: 15; Stmt: 16; Type: 17; Unmatched: 18
48	s85 : ;	–
49	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Equality: 36; Expr: 86; Mul: 41; Primary: 42; Relation: 43; Unary: 45
50	s87 : ;	–
51	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 88
52	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 89; Mul: 62; Primary: 63; Relation: 64; Unary: 66
53	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 90
54	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 91
55	r33 : !=,), <, <=, ==, >, >=; s92 : +; s93 : -	–
56	r48 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
57	r25 :); s94 : !=; s95 : ==	–

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
58	s96 :)	–
59	r47 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
60	r45 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
61	r46 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
62	r36 : !=,), +, -, <, <=, ==, >, >=; s97 : %; s98 : *; s99 : /	–
63	r44 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
64	r28 : !=,), ==; s100 : <; s101 : <=; s102 : >; s103 : >=	–
65	r49 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
66	r40 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
67	r43 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
68	s104 :)	–
69	r41 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
70	r42 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
71	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Mul: 105; Primary: 42; Unary: 45
72	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Mul: 106; Primary: 42; Unary: 45
73	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Mul: 41; Primary: 42; Relation: 107; Unary: 45
74	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Mul: 41; Primary: 42; Relation: 108; Unary: 45
75	r23 : ;, ID, char, float, if, int, return, while, {, }	–
76	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 109
77	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 110
78	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Primary: 42; Unary: 111
79	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 112; Mul: 41; Primary: 42; Unary: 45
80	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 113; Mul: 41; Primary: 42; Unary: 45
81	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 114; Mul: 41; Primary: 42; Unary: 45
82	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 115; Mul: 41; Primary: 42; Unary: 45
83	s116 :)	–
84	r2 : ;, ID, char, float, if, int, return, while, {, }	–
85	r22 : ;, ID, char, float, if, int, return, while, {, }	–
86	r11 : ;	–
87	r7 : ;, ID, char, float, if, int, return, while, {, }	–
88	r43 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
89	s117 :)	–

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
90	r41 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
91	r42 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
92	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Mul: 118; Primary: 63; Unary: 66
93	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Mul: 119; Primary: 63; Unary: 66
94	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Mul: 62; Primary: 63; Relation: 120; Unary: 66
95	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Mul: 62; Primary: 63; Relation: 121; Unary: 66
96	s122 : ;; s124 : ID; s128 : if; s129 : return; s130 : while; s131 : {	Block: 123; Matched: 125; Simple: 126; Stmt: 127; Unmatched: 18
97	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 132
98	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 133
99	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Primary: 63; Unary: 134
100	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 135; Mul: 62; Primary: 63; Unary: 66
101	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 136; Mul: 62; Primary: 63; Unary: 66
102	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 137; Mul: 62; Primary: 63; Unary: 66
103	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 138; Mul: 62; Primary: 63; Unary: 66
104	r50 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
105	r34 : !=, +, -, ;, <, <=, ==, >, >=; s76 : %; s77 : *; s78 : /	–
106	r35 : !=, +, -, ;, <, <=, ==, >, >=; s76 : %; s77 : *; s78 : /	–
107	r27 : !=, ;, ==; s79 : <; s80 : <=; s81 : >; s82 : >=	–
108	r26 : !=, ;, ==; s79 : <; s80 : <=; s81 : >; s82 : >=	–
109	r39 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
110	r37 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
111	r38 : !=, %, *, +, -, /, ;, <, <=, ==, >, >=	–
112	r29 : !=, ;, <, <=, ==, >, >=; s71 : +; s72 : -	–
113	r30 : !=, ;, <, <=, ==, >, >=; s71 : +; s72 : -	–
114	r31 : !=, ;, <, <=, ==, >, >=; s71 : +; s72 : -	–
115	r32 : !=, ;, <, <=, ==, >, >=; s71 : +; s72 : -	–
116	s12 : ID; s21 : if; s23 : return; s24 : while; s25 : {; s9 : ;	Block: 10; Matched: 139; Simple: 15; Unmatched: 140
117	r50 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
118	r34 : !=,), +, -, <, <=, ==, >, >=; s97 : %; s98 : *; s99 : /	–
119	r35 : !=,), +, -, <, <=, ==, >, >=; s97 : %; s98 : *; s99 : /	–

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
120	r27 : !=,), ==; s100 : <; s101 : <=; s102 : >; s103 : >=	–
121	r26 : !=,), ==; s100 : <; s101 : <=; s102 : >; s103 : >=	–
122	r24 : ;, ID, char, else, float, if, int, return, while, {, }	–
123	r16 : ;, ID, char, else, float, if, int, return, while, {, }	–
124	s141 : =	–
125	r13 : ;, ID, char, float, if, int, return, while, {, }; s142 : else	–
126	r15 : ;, ID, char, else, float, if, int, return, while, {, }	–
127	r19 : ;, ID, char, float, if, int, return, while, {, }	–
128	s143 : (	–
129	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Equality: 36; Expr: 144; Mul: 41; Primary: 42; Relation: 43; Unary: 45
130	s145 : (	–
131	r4 : ;, ID, char, float, if, int, return, while, {, }	Items: 146
132	r39 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
133	r37 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
134	r38 : !=, %,), *, +, -, /, <, <=, ==, >, >=	–
135	r29 : !=,), <, <=, ==, >, >=; s92 : +; s93 : -	–
136	r30 : !=,), <, <=, ==, >, >=; s92 : +; s93 : -	–
137	r31 : !=,), <, <=, ==, >, >=; s92 : +; s93 : -	–
138	r32 : !=,), <, <=, ==, >, >=; s92 : +; s93 : -	–
139	r18 : ;, ID, char, float, if, int, return, while, {, }	–
140	r21 : ;, ID, char, float, if, int, return, while, {, }	–
141	s30 : !; s31 : (; s32 : +; s33 : -; s35 : CHAR; s38 : FLOAT; s39 : ID; s40 : INT; s44 : STRING	Add: 34; Equality: 36; Expr: 147; Mul: 41; Primary: 42; Relation: 43; Unary: 45
142	s12 : ID; s21 : if; s23 : return; s24 : while; s25 : {; s9 : ;	Block: 10; Matched: 148; Simple: 15; Unmatched: 149
143	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 150; Mul: 62; Primary: 63; Relation: 64; Unary: 66
144	s151 : ;	–
145	s51 : !; s52 : (; s53 : +; s54 : -; s56 : CHAR; s59 : FLOAT; s60 : ID; s61 : INT; s65 : STRING	Add: 55; Equality: 57; Expr: 152; Mul: 62; Primary: 63; Relation: 64; Unary: 66
146	s12 : ID; s153 : }; s19 : char; s20 : float; s21 : if; s22 : int; s23 : return; s24 : while; s25 : {; s9 : ;	Block: 10; Decl: 11; Item: 13; Matched: 14; Simple: 15; Stmt: 16; Type: 17; Unmatched: 18
147	s154 : ;	–
148	r17 : ;, ID, char, float, if, int, return, while, {, }	–

Estado	ACTION: acción : terminales	GOTO: no terminal : estado
149	r20 : ;, ID, char, float, if, int, return, while, {, }	–
150	s155 :)	–
151	r23 : ;, ID, char, else, float, if, int, return, while, {, }	–
152	s156 :)	–
153	r2 : ;, ID, char, else, float, if, int, return, while, {, }	–
154	r22 : ;, ID, char, else, float, if, int, return, while, {, }	–
155	s122 : ;; s124 : ID; s128 : if; s129 : return; s130 : while; s131 : {	Block: 123; Matched: 157; Simple: 126; Stmt: 127; Unmatched: 18
156	s122 : ;; s124 : ID; s128 : if; s129 : return; s130 : while; s131 : {	Block: 123; Matched: 158; Simple: 126; Unmatched: 140
157	r13 : ;, ID, char, float, if, int, return, while, {, }; s159 : else	–
158	r18 : ;, ID, char, else, float, if, int, return, while, {, }	–
159	s122 : ;; s124 : ID; s128 : if; s129 : return; s130 : while; s131 : {	Block: 123; Matched: 160; Simple: 126; Unmatched: 149
160	r17 : ;, ID, char, else, float, if, int, return, while, {, }	–